Memory Poisoning of LLM Agents

The techniques to study for AegisBench's middle layer

Prepared as a study primer · 2026-10-26 · sources: MINJA, PMPA, MemSecBench, Injection–Execution Dissociation (full citations on last page)

---

0. The one idea to keep in your head

You are almost never changing the model's weights. You manipulate the **external memory** that the agent stores and later **retrieves and feeds back** into the model's context. The attack surface is the memory store (a JSON file, SQLite, or a vector DB) plus the retrieval step — not the LLM itself.

Every attack below can be decomposed into three questions you should measure separately:

- **Formation** — does the agent write the malicious content into persistent memory?
- **Retrieval** — does it pull that record back during a later task?
- **Behavioral influence** — does the retrieved record actually change the action it takes?

> **Why this beats the INJECT-* suite you already have**
>
> Your current injection challenges are single-shot, in-conversation prompts — the base model refuses them (you measured 0% followed across every config). The literature gets 50–85% success because it poisons a **persistent store** and triggers it in a **later, clean session**. That cross-session persistence is the whole reason this layer produces signal. It is the capability you have to build before the middle layer is real.

1. MINJA — the foundational attack (Dong et al., 2025)

MINJA (Memory INJection Attack) is the primary source the other three papers build on. Its threat model is deliberately weak-but-realistic: a **regular user with no elevated privileges** poisons the agent's long-term memory purely through **ordinary queries** — no access to the database or the framework. Reported >95% injection success and ~70% attack success under idealized conditions.

The three prompt-crafting primitives — learn these first:

- **Bridging steps** — insert intermediate reasoning that connects a benign-looking query to the malicious conclusion, so the agent 'derives' and stores the poisoned association itself.
- **Indication prompts** — cues embedded in the query that steer what the agent commits to memory as a reusable 'lesson' or few-shot example.
- **Progressive shortening** — start verbose to get the content stored, then shrink the trigger over turns until an innocuous phrase alone re-activates the poisoned memory.

> **Look into first**
>
> These three primitives are the actual mechanics of the attack. Everything else is a threat model or a benchmark wrapped around them. Implement MINJA-style query-only injection against a JSON store as your Experiment 1.

2. PMPA — persistent poisoning of harness-based agents (2609.13889)

The closest match to your setup: it attacks **harness-based agents** (OpenClaw, Claude Code) exactly like the harness you run. Two phases:

- **Injection phase** — the attacker publishes a benign-looking external source (text, PDF, or image) that embeds a **Conditional Malicious Rule**: an instruction specifying *when* to act maliciously. The agent reads it during a normal QA task and writes the rule into persistent memory on its own.
- **Trigger phase** — later, a **clean** workspace task whose context matches the rule's condition causes the agent to retrieve the poisoned memory and execute the malicious action (e.g. data exfiltration). The trigger prompt itself contains no malicious content.

Metrics worth adopting:

- **ISR** (Injection Success Rate) — did the malicious rule get written to memory.
- **C-ASR** (Cross-session Attack Success Rate) — did it fire in a later session.
- **Benign-task preservation** — does the agent still do legitimate work (guards against a useless attack).
- **C-ASR@i** — does it still fire after *i* rounds of benign use (persistence / durability).

> **Look into**
> The **Conditional Malicious Rule** payload design and the **cross-session trigger separation** are the ideas to port. Adopt ISR + C-ASR as your headline metrics. Skip the image/PDF carriers — your CTF agent is text-only.

3. MemSecBench — the lifecycle benchmark (2607.27080)

This is what AegisBench's middle layer should be positioned against. It turns poisoning into a measurable lifecycle rather than a single success/fail.

The Write–Execute–Forget protocol:

- **Write** — is the malicious instruction stored (a deterministic write-check).
- **Execute** — is it later recalled and turned into a real, verifiable consequence.
- **Forget** — can the poisoned entry be **selectively repaired** without wiping legitimate memory. This is your defense/clean-up metric.

Scored across seven evidence-based checkpoints, over a matrix of **harness × memory backend × LLM**. Crucially it compares memory **backends** (native file vs Mem0 vs A-Mem) and finds backend choice swings end-to-end attack success by up to 16 points and repair by up to 41 points.

> **Look into**
> Borrow the **Write–Execute–Forget protocol** and checkpoint scoring as your measurement scaffold. The backend comparison directly answers your JSON-vs-vector-DB question with data: start with a plain file store, and treat 'selective repair' as the defense arm.

4. Injection–Execution Dissociation — layered defenses (2605.08442)

The paper that most directly backs your 'depth of the stack' thesis — on the defense side. Its core insight: whether a model **stores** an injected instruction and whether it later **acts** on it are **separable properties**. Measure both; a defense can block one stage and not the other.

Six defenses tested across four architectural layers — and how they fared:

Layer	Defense	Result
Input	Input filters	Fail — never see the payload (it arrives via retrieval)
Retrieval	Retrieval classifiers	Fail at small sizes; frontier-model judges do flag it
Prompt	Prompt hardening	Fail — ~78% ASR, no real protection
Memory/tool	Memory Sandbox (remove recall tool)	Works — 0% ASR on 8/9 models

Even Memory Sandbox is bypassed by **RAG fallback**: a model rebuilds the instruction straight from the retrieval corpus when the recall tool is gone. The takeaway for your paper: **defense effectiveness rises as you move down the stack** — prompt-layer hardening is useless, memory-layer gating works. That is your thesis, demonstrated.

> **Look into**
> The **injection-vs-execution two-stage measurement** (it is the same formation/retrieval/influence split) and the **layered defense taxonomy**. Reproduce 'prompt-hardening fails, memory-gating works' on your local model and you have a self-contained result.

5. What to implement, in order

#	Technique	From	Why this order
1	Query-only injection: bridging steps, indication prompts, progressive shortening	MINJA	The attack mechanics. Nothing works without these.
2	Conditional Malicious Rule + cross-session trigger; ISR / C-ASR	PMPA	Turns the attack into your harness threat model + metrics.
3	Write–Execute–Forget protocol; file vs vector backend	MemSecBench	Your measurement scaffold; settles the storage choice.
4	Injection/execution split; layered defenses (incl. Memory Sandbox)	Leong	The defense arm that proves the depth thesis.

6. Wiring it into AegisBench

- **Add a persistent store** to the challenge runner — start with a JSON file that survives between challenge sessions. This is the single missing piece; your harness currently has no cross-session memory.
- **Add a retrieval hook** — before each task, select relevant stored records and prepend them to the prompt.
- **Run the three conditions** — (A) clean memory, (B) poisoned memory, (C) poisoned + a prompt-level safeguard — holding model, task, tools, and seed constant.
- **Seed realistic legitimate memory first.** The EHR study found pre-existing genuine records *dramatically* reduce attack success. Poisoning an empty store overstates the effect and a reviewer will catch it.
- **Run it on the 4b**, not the 9b — equal solve rate, faster, so you can afford the extra conditions (keep the 9b only for the comparison line).

> **The point of the whole exercise**
>
> Your prompt-layer (wrapper) result is a near-zero ceiling. If memory poisoning produces a large C-ASR and abliteration produces a larger one, you have the arc: **the deeper the intervention, the higher the ceiling.** This document is the toolkit for making the middle rung of that ladder measurable.

References

- MINJA — Dong et al., “A Practical Memory Injection Attack against LLM Agents”, 2025 (NeurIPS 2025). Verify the exact arXiv ID before citing.
- PMPA — Huang, Zhang, Jia, “When Malicious Instructions Persist: Persistent Memory Poisoning Attack on Harness-Based Agents”, arXiv:2609.13889 (Sep 2026). Preprint, under review.
- MemSecBench — Chen et al., “MemSecBench: Tracking Agent Memory Poisoning from Persistence to Consequence and Repair”, arXiv:2607.27080 (Jul 2026).
- Injection–Execution Dissociation — Leong, “A Mechanistic Evaluation of Persistent Memory Attacks and Defenses in Stateful LLM Agents”, arXiv:2605.08442 (v6, Oct 2026). Cite the latest version.
- Memory Poisoning Attack and Defense on Memory-Based LLM-Agents — Devarangadi Sunil et al., arXiv:2601.05504 (Jan 2026). UMass course project report, not peer-reviewed — weight accordingly.

Caveat: all attack-success figures above are author-reported and mostly on frontier/hosted models in non-security domains (healthcare, email, office). None run on a local small model in a pentest/CTF harness — which is exactly your

contribution gap.